Load Balancing :

Input : m identical machines; n jobs, job j has processing time t_j.

→ Job j must run at a stretch on one machine.
→ A machine can process at most one job at a time.

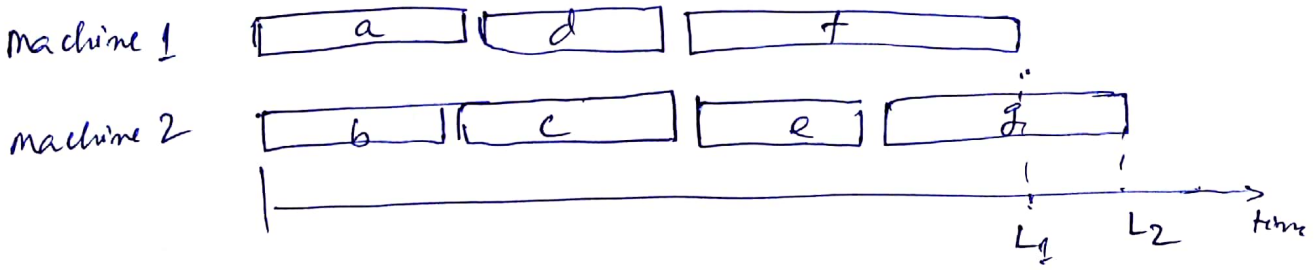
Def: Let $J(i)$ be the subset of jobs assigned to machine i.

The load of machine i is $L_i = \sum_{j \in J(i)} t_j$.

Def: The makespan is the maximum load to any machine

$$L = \max_i L_i .$$

Load balancing : Assign each job to a machine to minimize makespan.



makespan = L_2

Claim: Load balancing is NP-hard even if only 2 machines.

Greedy: List-scheduling algorithm

→ Consider n jobs in some fixed order.
→ Assign job j to machine whose load is smallest so far.

```
List-Scheduling (m, n, t1, t2, ... tn) {
    for i = 1 to m {
        Li = 0          ← load on machine i
        J(i) = φ        ← jobs assign to machine i
    }
    for j = 1 to n {
        i = argmin_k Lk     ← machine i has smallest load.
        J(i) ← J(i) ∪ {j}   ← assign job j to machine i.
        Li ← Li + tj        ← Update load of machine i.
    }
    return J(1), ... J(m).
```

Implementation: $O(n \log m)$ using priority queue.

Theorem: [Graham 1966] Greedy algorithm is a 2-approximation.
→ First worst case analysis of an approximation algorithm.
→ Need to compare resulting solution with optimal makespan L^*

Lemma 1: The optimal makespan $L^* \geq \max_j t_j$

Proof: Some machine must process the most time consuming job.

Lemma 2: The optimal makespan $L^* \geq \frac{1}{m} \sum_j t_j$

Proof: The total processing time is $\sum_j t_j$

one of m machine must do at least a $1/m$ fraction of total work.

Theorem: Greedy algorithm is a 2-approximation.

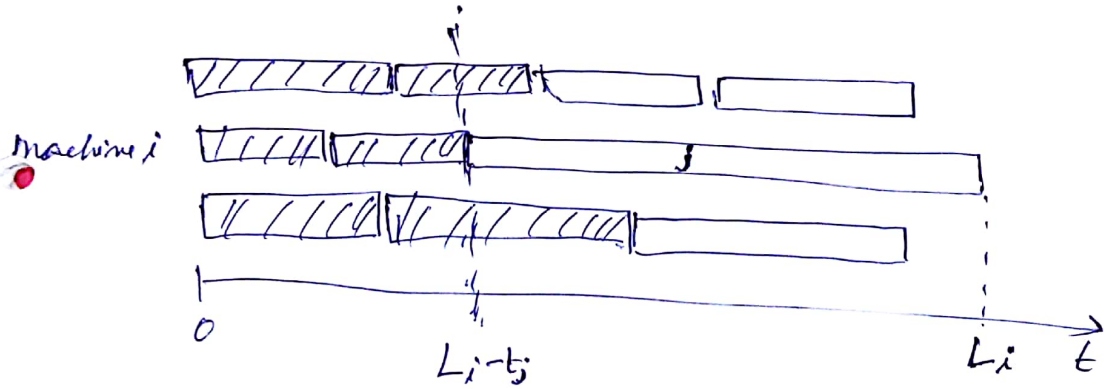
Proof: Consider load L_i of bottleneck machine i.

→ Let j be last job scheduled on machine i

→ When job j assigned to machine i, i had smallest load,

→ Its load before assignment is

$$L_i - t_j \Rightarrow L_i - t_j \leq L_k \text{ for all } 1 \leq k \leq m.$$



So, $$L_i - t_j \leq \frac{1}{m} \sum_k L_k$$

$$= \frac{1}{m} \sum_k t_k$$

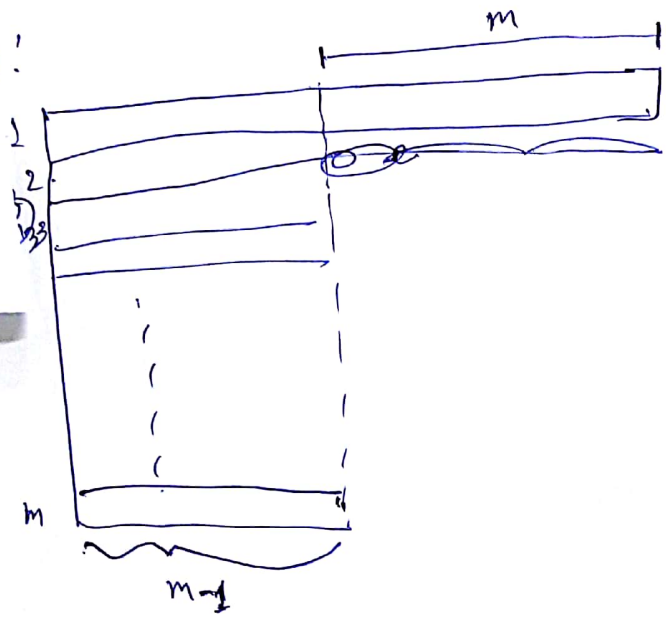
$$\leq L^* \quad \leftarrow \text{Lemma 2.}$$

→ Now $L_i = \underbrace{(L_i - t_j)}_{\leq L^*} + \underbrace{t_j}_{\leq L^*} \leq 2L^*$ □

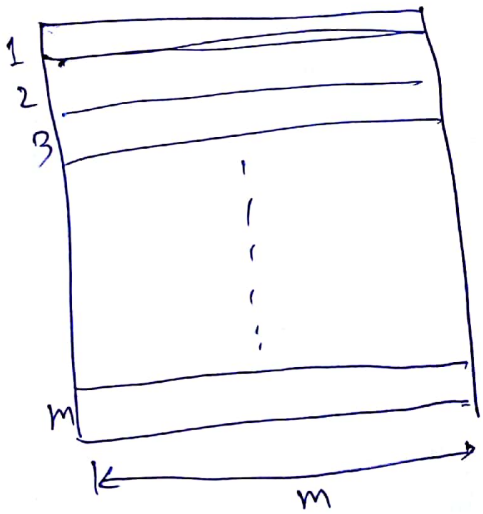
Is this analysis right?

Ex: m machine, m(m-1) jobs length 1 jobs, One job of length m.

Greedy!



Optimal Solution:



So $\rho = \frac{2m-1}{m} \simeq 2$

Longest Processing time (LPT): Sort n jobs in descending order of processing time, and then run list scheduling algorithm.

Observation: If at most m jobs, then list scheduling is optimal.

Pf:- Each job put on its own machine.

Lemma 3: If there are more than m jobs, $L^* \geq 2\, t_{m+1}$

Proof:- Consider first $m+1$ jobs $t_1, t_2 \ldots, t_{m+1}$.

→ Since t_i's are in descending order, each takes at least t_{m+1} time

→ There are $m+1$ jobs and m machines, so by pigeonhole principle, at least one machine gets two jobs.

$$\Rightarrow \quad L^* \geq 2\, t_{m+1}$$

$$\Rightarrow \quad t_{m+1} \leq \tfrac{1}{2} L^*$$

Theorem: LPT rule is a 3/2 - approximation algorithm.

Proof:

$$L_i = \underbrace{(L_i - t_j)}_{\leq L^*} + \underbrace{t_j}_{\leq \frac{1}{2}L^*} \leq \frac{3}{2} L^*$$

↑ Lemma 3.

There is a more tighter result ⇒ LPT is 4/3 approximation

Polynomial-time approximation Scheme (PTAS):- $(1+\varepsilon)$ - approximation algorithm for any constant $\varepsilon > 0$.

Consequence: PTAS produces arbitrarily high quality solution, but trades off accuracy for time.

FPTAS - If the approximation scheme's running time is polynomial in both $1/\varepsilon$ and n.

Subset-Sum Problem!

Input: A set of positive integer $S = \{x_1, x_2, \cdots x_n\}$ and a positive integer t.

Output: A subset of S whose sum is as large as possible, but not larger than t.

L_i - the list of sums of all subsets of $\{x_1, x_2, \cdots x_i\}$ that do not exceed t.

Def: $L = \{l_1, l_2, \cdots l_m\}$

Then $L + x = \{l_1 + x, l_2 + x, \cdots l_m + x\}$

$= \{l + x : l \in L\}$

Def: Merge-List (L, L') returns the sorted merge list of L and L' with duplicates removed

↑ they are also sorted

Exact-Subset-Sum (S,t)

$n = |S|$

$L_0 = \{0\}$

for $i = 1$ to n

L_i = Merge-Lists $(L_{i-1}, L_{i-1} + x_i)$

remove from L_i every element that is greater than t.

return the largest element in L_n

Length of L_i can be as much as 2^i

An FPTAS for Subset-Sum

tricks: Trimming each list L_i after it is created.

If there are two close values in L, then we discard one.

Trimming parameter → δ where $0 < \delta < 1$.

For each y removed from L, there is an element z in L, such that

z ↑ approximate y

$$\frac{y}{1+\delta} \leq z \leq y$$

Ex:- $\delta = 0.1$

$L = \{10, 11, 12, 15, 20, 21, 22, 23, 24, 29\}$

$L' = \{10, 12, 15, 20, 23, 29\}$

Assume L is sorted in monotonically increasing order.

$L = \{y_1, y_2, \ldots, y_m\}$

```
Trim (L, δ)
    let m ← |L|
    L' = {y_1}
    last = y_1
    for i = 2 to m
        if y_i > last · (1+δ)
            append y_i onto the end of L'
            last = y_i
    return L'.
```

```
Approx-Subset-Sum (S, t, ε)
    n = |S|
    L_0 = {0}
    for i = 1 to n
        L_i = Merge-List (L_{i-1}, L_{i-1} + x_i)
        L_i = Trim (L_i, ε/2n)
        remove elements greater than t from L_i
    let z* be the largest value in L_n
    return z*
```

Example: $S = \{104, 102, 201, 101\}$

$t = 308$

$\epsilon = 0.40 \Rightarrow \delta = \epsilon/8 = 0.05$

$L_0 = \{0\}$

Pass 1: line 4: $L_1 = \{0, 104\}$
line 5: $L_1 = \{0, 104\}$
line 6: $L_1 = \{0, 104\}$

Pass 2: Line 4: $L_2 = \{0, 102, 104, 206\}$
line 5: $L_2 = \{0, 102, 206\}$
line 6: $L_2 = \{0, 102, 206\}$

Pass 3: Line 4: $L_3 = \{0, 102, 201, 206, 303, 407\}$
line 5: $L_3 = \{0, 102, 201, 303, 407\}$
line 6: $L_3 = \{0, 102, 201, 303\}$

Pass 4: line 4: $L_4 = \{0, 101, 102, 201, 203, 302, 303, 404\}$
line 5: $L_4 = \{0, 101, 201, 302, 404\}$
line 6: $L_4 = \{0, 101, 201, 302\}$

So $z^* = 302$

Optimal $= 307 = 104 + 102 + 101$

$\frac{\text{Optimal}}{z^*} < 1 + \epsilon$

Theorem: Approx-Subset-Sum is a FPTAS for the subset-sum problem. with $0 < \epsilon < 1$

Proof: Let y^* is the optimal value.

We have to show that $\frac{y^*}{z^*} \leq 1 + \epsilon$

And running time is polynomial in both $1/\epsilon$ and n.

For every discarded $y \in L_i$

there is an $z \in L_i$ such that

$$\frac{y}{(1+\epsilon/2n)^i} \leq z \leq y$$

So, $$\frac{y^*}{(1+\epsilon/2n)^n} \leq z \leq y^*$$

$$\Rightarrow \frac{y^*}{z} \leq \left(1 + \frac{\epsilon}{2n}\right)^n$$

Since z^* is the max element in L'_n.

$$\frac{y^*}{z^*} \leq \left(1 + \frac{\epsilon}{2n}\right)^n.$$

$$\Rightarrow \frac{y^*}{z^*} \leq e^{\epsilon/2} \qquad \left[\text{since } \lim_{n\to\infty}\left(1+\frac{x}{n}\right)^n = e^x\right]$$

When $|x| \leq 1$, we have $1+x \leq e^x \leq 1+x+x^2$

we have $e^{\epsilon/2} \leq 1 + \epsilon/2 + (\epsilon/2)^2$

$\leq 1+\epsilon$ since $0 < \epsilon < 1$

So, $\frac{y^*}{z^*} \leq 1+\epsilon$.

To show that this algorithm is FPTAS, we have to derive a bound on the length of L_i.

Since Trim(L, δ) runs for the size of L, so if size of L is polynomial, then we are done.

After trimming, successive elements z, z' of L_i must have the relationship $\frac{z'}{z} \geq 1 + \epsilon/2n$.

So, Each list contains the value 0, possibly 1, and upto $\lfloor \log_{1+\epsilon/2n} t \rfloor$ additional values. [∵ They differ by a factor of at least $1+\epsilon/2n$]

So, The max size of L_i is

$$\log_{1+\epsilon/2n} t + 2 = \frac{\ln t}{\ln(1+\epsilon/2n)} + 2$$

$$\leq \frac{2n(1+\epsilon/2n)\ln t}{\epsilon} + 2 \quad \left[\because \frac{x}{1+x} \leq \ln(1+x) \leq x \text{ when } x > -1\right]$$

$$< \frac{4n \ln t}{\epsilon} + 2 \quad [\text{since } \epsilon/2n = \delta < 1]$$